Complete Notes: 8259A PIC and x86 Instruction Execution

(8086 / 80386 Assembly Language)

Block Diagram: CPU with Master and Slave 8259A PICs

Question

Draw a block diagram showing the connection between a CPU, a master 8259A, and two slave 8259As. Clearly label the INT, INTR, IR, and CAS lines.

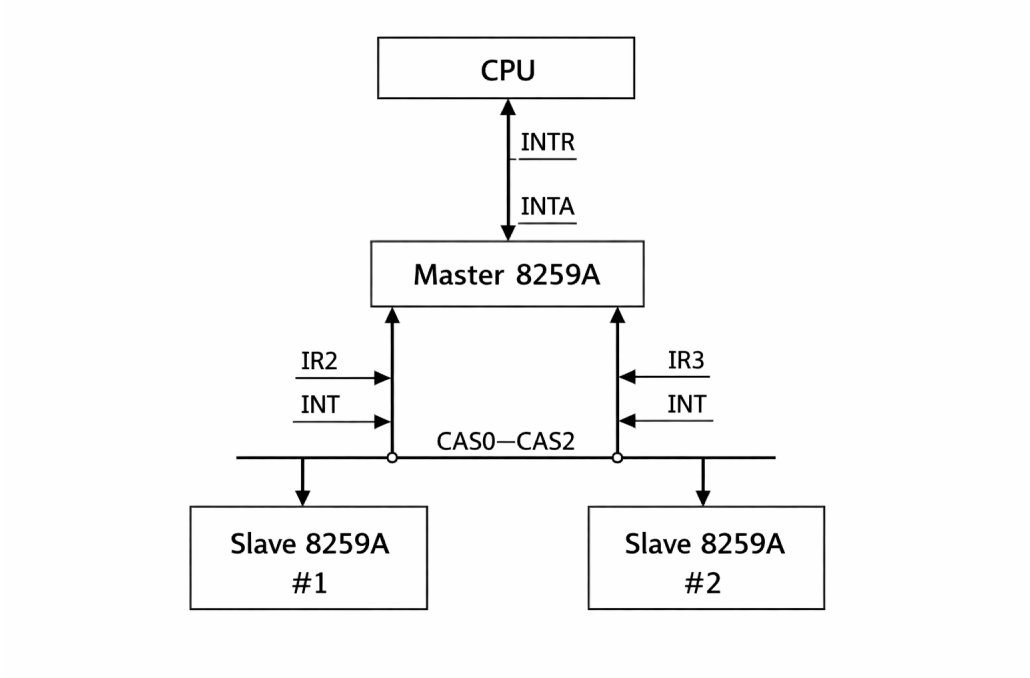

Figure 1: CPU connected with Master and Slave 8259A PICs

Solution

Explanation:

- The **Master 8259A** directly connects to the CPU.
- The **INT** output of the master PIC connects to the CPU **INTR** pin.
- Each **Slave 8259A** connects its **INT** output to one of the **IR** inputs of the master PIC.
- The **CAS0–CAS2 (Cascade Lines)** are driven by the master and connected to all slaves to identify which slave is requesting service.

Signal Meaning:

- **IR0–IR7**: Interrupt request lines
- **INT**: Interrupt output of PIC

- **INTR**: Interrupt input to CPU
- **CAS0–CAS2**: Cascade identification lines

Execution of Assembly Instructions

Question

Explain the execution of the following instructions.

(a) `mov cx, cs`

Solution

Copies the contents of the Code Segment register (CS) into CX. Flags are unaffected.

(b) `mov ax, [alpha]`

Solution

Loads a 16-bit value from memory at address `DS:alpha` into AX.

(c) `xchg bx, cx`

Solution

Exchanges the contents of BX and CX registers. Flags are not affected.

(d) `lea op1, op2`

Solution

Loads the **effective address** of operand `op2` into `op1`. No memory access is performed.

(e) `lds bx, [4326]`

Solution

Loads a far pointer from memory:

- BX ← word at 4326H
- DS ← word at 4328H

(f) `inc ebx`

Solution

Increments EBX by 1. Updates all flags except CF.

(g) `dec dl`

Solution

Decrements DL by 1. CF is not affected.

(h) `inc [count]`

Solution

Increments the memory variable `count` by 1.

(i) `add eax, ebx`

Solution

Adds EBX to EAX and stores the result in EAX. All arithmetic flags are updated.

(j) `mul dl`

Solution

Unsigned multiplication:

$$AX = AL \times DL$$

CF and OF are set if AH $\neq 0$.

(k) `mul dx`

Solution

Unsigned multiplication:

$$DX : AX = AX \times DX$$

CF and OF are set if DX $\neq 0$.

(l) `div bl`

Solution

Unsigned division:

$$AX \div BL$$

Quotient $\rightarrow$ AL, Remainder $\rightarrow$ AH.

(m) `div bx`

Solution

Unsigned division:

$$DX : AX \div BX$$

Quotient → AX, Remainder → DX.

(n) `adc op1, op2`

Solution

Adds operands including carry flag:

$$op1 = op1 + op2 + CF$$

(o) `sbb op1, op2`

Solution

Subtracts operand and borrow:

$$op1 = op1 - (op2 + CF)$$

(p) ASCII Addition with AAA

Solution

Code:

```
sub ah, ah
mov al, '6'
add al, '7'
aaa
```

Detailed Explanation with Exact Calculations:

- `sub ah, ah` clears the AH register:

$$AH = 00H$$

- `mov al, '6'` loads the ASCII value of character '6' into AL:

$$'6' = 36H \Rightarrow AL = 36H$$

- `add al, '7'` adds the ASCII value of '7':

$$'7' = 37H$$

$$AL = 36H + 37H = 6DH$$

- The lower nibble of AL is:

$$6DH = 0110\,1101_2 \Rightarrow \text{Lower nibble} = DH = 13$$

  Since $13 > 9$, the result is not a valid decimal digit and `AAA` must adjust it.

- **AAA Adjustment Steps:**
  - **Step 1: Add 06H to AL**

$$AL = 6DH + 06H = 73H$$

  - **Step 2: Increment AH to store decimal carry**

$$AH = 00H + 01H = 01H$$

  - **Step 3: Mask AL to keep only the units digit**

$$AL = 73H \;\&\; 0FH = 03H$$

Final Result:

$$AH = 01H, \quad AL = 03H$$

This corresponds to the decimal value **13** in **unpacked BCD format**, where AH holds the tens digit and AL holds the units digit.

(q) Packed BCD Addition with DAA

Solution

Code:

```
mov al, 53
mov cl, 29
add al, cl
daa
```

Detailed Explanation with Exact Calculations:

- The values `53` and `29` are treated as **packed BCD numbers**.
  - 53H represents decimal 53
  - 29H represents decimal 29
- `add al, cl` performs binary addition:

$$AL = 53H + 29H = 7CH$$

- Binary representation of the result:

$$7CH = 0111\,1100_2$$

  The lower nibble is:

$$CH = 12\ (> 9)$$

  Hence, the result is not a valid packed BCD digit.

- The instruction `DAA` (Decimal Adjust after Addition) corrects the result to valid packed BCD.

- **DAA Adjustment Rules Applied:**

  – Since the lower nibble > 9, `DAA` adds 06H to AL:

$$AL = 7CH + 06H = 82H$$

  – The upper nibble does not exceed 9 and no carry is generated beyond 99, so no 60H adjustment is needed.

Final Result:

$$AL = 82H$$

This corresponds to the correct packed BCD representation of the decimal value:

$$53 + 29 = 82$$

(r) Packed BCD Addition with Carry

Solution

Code:

```
mov al, 73
mov cl, 29
add al, cl
daa
```

Detailed Explanation with Exact Calculations:

- The values `73` and `29` are treated as **packed BCD numbers**.

  – 73H represents decimal 73

  – 29H represents decimal 29

- `add al, cl` performs binary addition:

$$AL = 73H + 29H = 9CH$$

- Binary form of the result:

$$9CH = 1001\,1100_2$$

  The lower nibble is:

$$CH = 12\ (> 9)$$

  Hence, the lower decimal digit is invalid and requires correction.

- **DAA Step 1: Lower Nibble Adjustment**
  - Since the lower nibble > 9, `DAA` adds 06H to AL:

$$AL = 9CH + 06H = A2H$$

- After this correction, the value of AL becomes:

$$A2H = 1010\,0010_2$$

  The upper nibble is now:

$$AH = 10\ (> 9)$$

  This indicates a decimal carry beyond 99.

- **DAA Step 2: Upper Nibble (Decimal Carry) Adjustment**
  - Since the result exceeds 99, `DAA` adds 60H:

$$AL = A2H + 60H = 02H$$

  - A carry is generated, so the Carry Flag is set:

$$CF = 1$$

Final Result:

$$AL = 02H, \quad CF = 1$$

This represents the correct packed BCD result of:

$$73 + 29 = 102$$

The value `02H` is stored in AL, and the hundreds carry is indicated by CF = 1.

(s) ASCII Subtraction with AAS

Solution

Code:

```
sub ah, ah
mov al, '9'
```

```
sub al, '3'
aas
```

Detailed Explanation with Exact Calculations:

- `sub ah, ah` clears AH:

$$AH = 00H$$

  AH is cleared because in unpacked BCD operations AH may be used to hold a borrow/carry.

- `mov al, '9'` loads the ASCII value of character '9' into AL:

$$'9' = 39H \Rightarrow AL = 39H$$

- `sub al, '3'` subtracts the ASCII value of '3':

$$'3' = 33H$$

$$AL = 39H - 33H = 06H$$

- Check the lower nibble after subtraction:

$$06H \Rightarrow \text{lower nibble} = 6 \ (\leq 9)$$

  Also, there is **no borrow** from the lower nibble (AF = 0).

- **AAS rule:** Adjustment is performed only if the lower nibble is invalid (i.e., > 9) or if a borrow occurred (AF = 1). Since neither condition holds here, `AAS` performs **no adjustment**.

Final Result:

$$AH = 00H, \quad AL = 06H$$

This represents the correct decimal subtraction:

$$9 - 3 = 6$$

in unpacked BCD form (AL holds the units digit).

(t) ASCII Subtraction with Borrow

Solution

Code:

```
sub ah, ah
mov al, '3'
sub al, '9'
aas
```

Detailed Explanation with Exact Calculations:

- `sub ah, ah` clears AH:
$$AH = 00H$$
- `mov al, '3'` loads ASCII value of '3' into AL:
$$'3' = 33H \Rightarrow AL = 33H$$
- `sub al, '9'` subtracts ASCII value of '9':
$$'9' = 39H$$
$$AL = 33H - 39H = FAH$$
Here, a **borrow** occurs because $3 < 9$.
- Check the lower nibble:
$$FAH \Rightarrow \text{lower nibble} = AH = 10\ (> 9)$$
This is invalid for a decimal digit. Also, a borrow from the low nibble sets AF = 1. Hence, `AAS` must adjust the result.
- **AAS Adjustment Steps:**
  - **Step 1: Subtract 06H from AL**
$$AL = FAH - 06H = F4H$$
  - **Step 2: Subtract 01H from AH (borrow propagation)**
$$AH = 00H - 01H = FFH$$
This indicates a borrow into the next decimal digit.
  - **Step 3: Mask AL to keep only units digit**
$$AL = F4H \ \& \ 0FH = 04H$$
- After adjustment, `AAS` sets:
$$CF = 1, \quad AF = 1$$
to indicate that a borrow occurred.

Final Result:
$$AH = FFH, \quad AL = 04H, \quad CF = 1$$
This means the subtraction produced a **negative result**. AL contains the corrected units digit (4), and CF=1 indicates borrow (negative).

(u) Packed BCD Subtraction with DAS

Solution

Code:

```
mov al, 75
mov bh, 46
sub al, bh
das
```

Detailed Explanation with Exact Calculations:

- The values are treated as **packed BCD numbers**:
  - 75H represents decimal 75
  - 46H represents decimal 46
- Perform binary subtraction:

$$AL = 75H - 46H = 2FH$$

- Analyze the result:

$$2FH = 0010\,1111_2$$

  The lower nibble is:

$$FH = 15\ (> 9)$$

  A packed BCD digit must be in the range 0–9, so the lower digit is invalid.

- **DAS Rule (lower digit correction):** If the lower nibble > 9 **or** AF $= 1$, then subtract 06H.
- Apply DAS correction on the lower nibble:

$$AL = 2FH - 06H = 29H$$

- Since the upper nibble is now valid (2) and no borrow beyond 99 occurred, no 60H correction is needed.

Final Result:

$$AL = 29H$$

This is the correct packed BCD result of:

$$75 - 46 = 29$$

(v) Packed BCD Subtraction with Borrow

Solution

Code:

```
mov al, 38
mov ch, 61
sub al, ch
das
```

Detailed Explanation with Exact Calculations:

- The values are treated as **packed BCD numbers**:
  - 38H represents decimal 38
  - 61H represents decimal 61
- Perform binary subtraction:

$$AL = 38H - 61H = D7H$$

  A borrow occurs because $38 < 61$, so:

$$CF = 1$$

- Analyze the lower nibble:

$$D7H = 1101\,0111_2 \Rightarrow \text{lower nibble} = 7\ (\leq 9)$$

  So no 06H correction is required for the lower digit (assuming AF = 0).

- **DAS Rule (upper digit / borrow correction):** If CF = 1 (borrow beyond the tens place), then `DAS` subtracts 60H to correct the packed BCD result.
- Apply the 60H correction:

$$AL = D7H - 60H = 77H$$

  CF remains set to indicate a borrow:

$$CF = 1$$

Final Result:

$$AL = 77H, \quad CF = 1$$

Interpretation: In packed BCD arithmetic, CF=1 indicates the result is negative. The magnitude corresponds to:

$$38 - 61 = -23$$

and `77H` is the 10's-complement representation of 23 in two BCD digits (i.e., $100 - 23 = 77$).

(w) AAM after Multiplication

Solution

Code:

```
mov al, 3
mov bl, 9
mul bl
aam
```

Detailed Explanation with Exact Calculations:

- `mov al, 3` loads AL with 03H and `mov bl, 9` loads BL with 09H.
- `mul bl` performs **unsigned 8-bit multiplication**:

$$AX = AL \times BL$$

$$AX = 03H \times 09H = 1BH$$

Since the product fits in 8 bits, the result is:

$$AX = 001BH \Rightarrow AH = 00H,\ AL = 1BH$$

and:

$$1BH = 27_{10}$$

- `AAM` (ASCII Adjust after Multiply) converts the binary result in AL into **two unpacked decimal digits**:

$$AH = \left\lfloor \frac{AL}{10} \right\rfloor, \quad AL = AL \bmod 10$$

- Apply the formula for AL = 27:

$$AH = \left\lfloor \frac{27}{10} \right\rfloor = 2, \quad AL = 27 \bmod 10 = 7$$

Final Result:

$$AH = 02H, \quad AL = 07H$$

This represents the decimal number **27** in **unpacked format**, where AH holds the tens digit and AL holds the units digit.

(x) ASCII Multiplication with AAM

Solution

Code:

```
mov al, '3'
mov bl, '9'
```

```
and al, 0fh
and bl, 0fh
mul bl
aam
```

Detailed Explanation with Exact Calculations:

- `mov al, '3'` loads ASCII code of '3' into AL and `mov bl, '9'` loads ASCII code of '9' into BL:
$$'3' = 33H \Rightarrow AL = 33H, \qquad '9' = 39H \Rightarrow BL = 39H$$

- The instructions `and al, 0fh` and `and bl, 0fh` convert ASCII digits into their numeric values by keeping only the lower nibble:
$$AL = 33H \ \& \ 0FH = 03H$$
$$BL = 39H \ \& \ 0FH = 09H$$
Reason: For ASCII digits '0'–'9', the lower nibble already equals the numeric value.

- `mul bl` performs unsigned 8-bit multiplication:
$$AX = AL \times BL$$
$$AX = 03H \times 09H = 1BH$$
Thus:
$$AX = 001BH \Rightarrow AL = 1BH = 27_{10}$$

- `AAM` converts the binary result in AL into two unpacked decimal digits:
$$AH = \left\lfloor \frac{AL}{10} \right\rfloor, \quad AL = AL \bmod 10$$
For $AL = 27$:
$$AH = 2, \quad AL = 7$$

Final Result:
$$AH = 02H, \quad AL = 07H$$

This represents the decimal number **27** in unpacked form (AH = tens digit, AL = units digit).

(y) AAD before Division

Solution

Code:

```
mov ax, 0207h
mov bl, 05h
aad
div bl
```

Detailed Explanation with Exact Calculations:

- `mov ax, 0207h` loads:

$$AH = 02H, \quad AL = 07H$$

  This represents an **unpacked decimal number** where AH holds the tens digit (2) and AL holds the units digit (7), i.e., "27".

- `aad` (ASCII Adjust before Division) converts this unpacked form into a **binary number** using:

$$AL = (AH \times 10) + AL, \qquad AH = 00H$$

  Apply the formula:

$$AL = (02H \times 0AH) + 07H = 14H + 07H = 1BH$$

  Thus after `aad`:

$$AH = 00H, \quad AL = 1BH$$

  and:

$$1BH = 27_{10}$$

- `div bl` performs **unsigned 8-bit division**:

$$\text{Dividend} = AX, \quad \text{Divisor} = BL$$

  Here:

$$AX = 001BH = 27, \qquad BL = 05H = 5$$

  Division:

$$27 \div 5 = 5 \text{ (quotient)}, \quad 2 \text{ (remainder)}$$

- For `div bl`:
  - Quotient $\rightarrow$ AL
  - Remainder $\rightarrow$ AH

Final Result:

$$AL = 05H \text{ (Quotient)}, \quad AH = 02H \text{ (Remainder)}$$

Rotate and Carry Instructions

(a) ROL Instruction (Rotate Left)

Solution

Code:

```
mov al, 11110000b
rol al, 1
```

Detailed Explanation:

- Initial value in AL:
$$AL = 11110000_2 = F0H$$
- `ROL` rotates all bits to the left by one position.
  - The most significant bit (MSB) is shifted out.
  - The MSB is inserted back into the least significant bit (LSB).
  - The MSB is also copied into the Carry Flag (CF).
- Bit movement:

Before: 1 1110000

After: 1110000 1

Final Result:
$$AL = 11100001_2 = E1H, \quad CF = 1$$

(b) ROR Instruction (Rotate Right)

Solution

Code:

```
mov dl, 3fh
ror dl, 4
```

Detailed Explanation:

- Initial value in DL:
$$DL = 00111111_2 = 3FH$$
- `ROR` rotates bits to the right.
- Rotating right by 4 positions is equivalent to performing four single-bit right rotations.
- Bits shifted out from the LSB re-enter at the MSB position.

Rotation result:
$$00111111_2 \xrightarrow{\text{ROR } 4} 11110011_2$$

Final Result:

$$DL = F3H$$

(c) RCL Instruction (Rotate through Carry Left)

Solution

Code:

```
clc
mov bl, 88h
rcl bl, 1
rcl bl, 1
```

Detailed Explanation:

- `clc` clears the Carry Flag:
$$CF = 0$$
- Initial value in BL:
$$BL = 10001000_2 = 88H$$
- `RCL` rotates bits left through the Carry Flag.
  - CF becomes part of the rotation.
  - MSB goes into CF.
  - Old CF goes into LSB.
- **First RCL:**
$$CF = 0, \quad BL = 10001000$$
$$\Rightarrow BL = 00010000_2 = 10H, \quad CF = 1$$
- **Second RCL:**
$$CF = 1, \quad BL = 00010000$$
$$\Rightarrow BL = 00100001_2 = 21H, \quad CF = 0$$

Final Result:

$$BL = 21H, \quad CF = 0$$

(d) RCR Instruction (Rotate through Carry Right)

Solution

Code:

```
stc
mov ah, 10h
```

`rcr ah, 1`

Detailed Explanation:

- `stc` sets the Carry Flag:
$$CF = 1$$
- Initial value in AH:
$$AH = 00010000_2 = 10H$$
- `RCR` rotates bits right through the Carry Flag.
  - CF is inserted into the MSB.
  - LSB is moved into CF.
- Bit movement:
$$CF \mid AH = 1\ 00010000$$
$$\Rightarrow AH = 10001000_2 = 88H, \quad CF = 0$$

Final Result:
$$AH = 88H, \quad CF = 0$$

Rotate and Carry Instructions

ROL and ROR

Solution

Rotate bits within a register without involving carry.

RCL and RCR

Solution

Rotate bits through the Carry Flag. Carry becomes part of rotation.

Program Execution: REPNE SCASB

Question

Explain operation of each instruction of the following program.

Solution

- `MOV CX,100H`: Initializes counter for 256 bytes.
- `MOV AL,0AH`: Search key.

- `CLD`: Clears direction flag so DI increments.
- `REPNE SCASB`: Repeats comparison until match or CX=0.
- `JCXZ NOT_FOUND`: Jumps if CX becomes zero.
- `STC`: Indicates value found (Carry Flag = 1).

Conclusion:

- CF = 1 ⇒ Value Found
- CF = 0 ⇒ Value Not Found